

Machine Learning for Systems and Control

5SC28

Lecture 7

dr. ir. Maarten Schoukens & prof. dr. ir. Roland Tóth

Control Systems Group

Department of Electrical Engineering

Eindhoven University of Technology

Academic Year: 2025-2026 (18/05/2026)

Reinforcement Learning

Overview of RL

Approximate TD Learning for Prediction

Approximate TD Learning for Control

Deep Q-Networks

Gaussian Process Q-Learning

Reinforcement Learning

Overview of RL

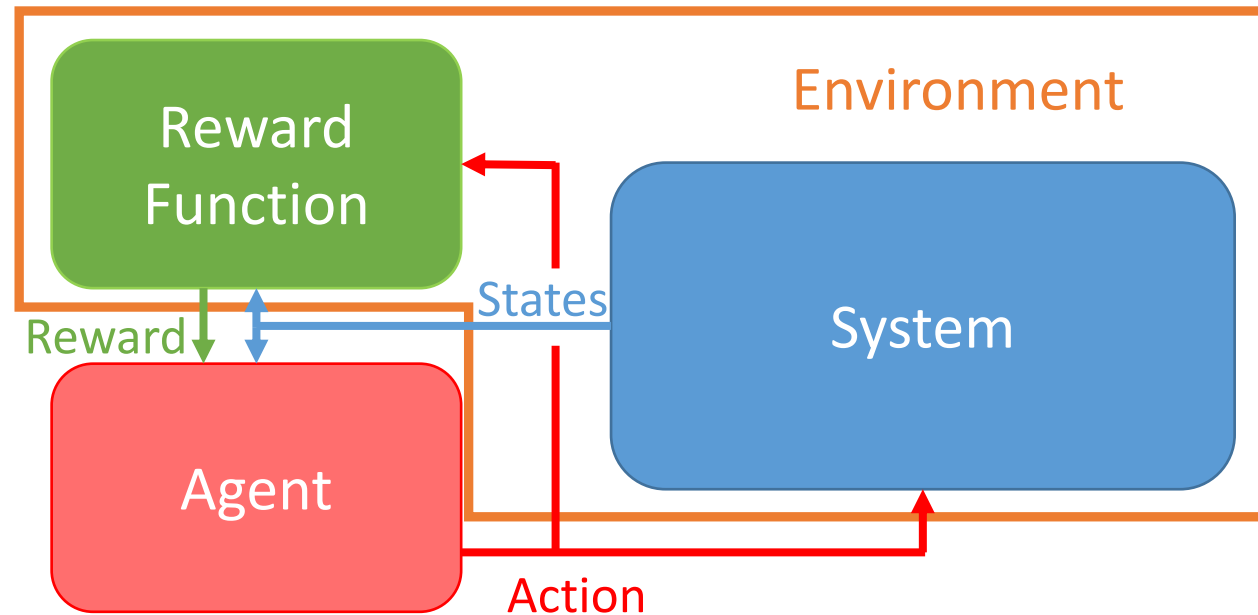
Approximate TD Learning for Prediction

Approximate TD Learning for Control

Deep Q-Networks

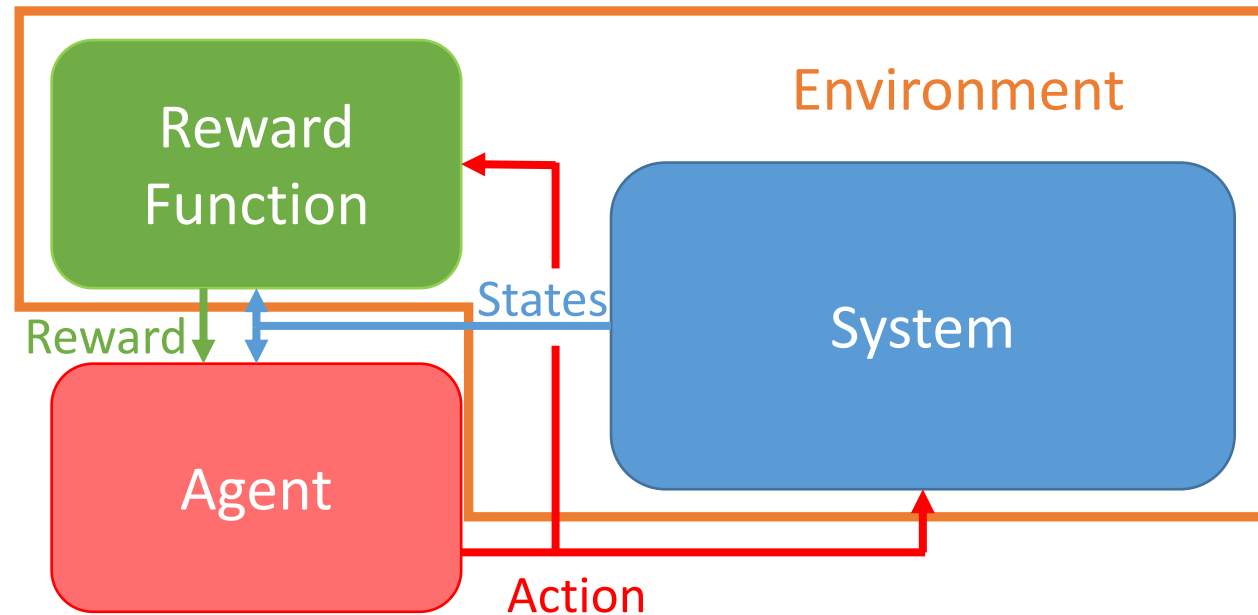
Gaussian Process Q-Learning

Principle of Reinforcement Learning (Recap)



- Agent interacts with the system through **States** and **Actions**
- Receive **Reward** as a performance feedback

Principle of Reinforcement Learning (Recap)



- Agent interacts with the system through **States** and **Actions**
- Receive **Reward** as a performance feedback
- This lecture: **approximate RL** – continuous states & actions

Reinforcement Learning

Overview of RL

Approximate TD Learning for Prediction

Approximate TD Learning for Control

Deep Q-Networks

Gaussian Process Q-Learning

Important ingredients

Discounted Return: $G_k = \mathbb{E}_\pi \left\{ \sum_{\tau=0}^{\infty} \gamma^\tau r_{k+\tau+1} \right\}$

Value Function: $V_\pi(x_k) = \mathbb{E}_\pi \left\{ \sum_{\tau=0}^{\infty} \gamma^\tau r_{k+\tau+1} \middle| x_k \right\}$

Q-Function: $Q_\pi(x_t, u_t) = \mathbb{E}_\pi \left\{ \sum_{\tau=0}^{\infty} \gamma^\tau r_{k+\tau+1} \middle| x_k, u_k \right\}$

Expected discounted return given the system states and the action taken at time k for policy π

The Prediction Problem

Estimate the Value Function $V_\pi(x)$ for a given policy π .

New Estimate $\leftarrow$ Old Estimate + Step Size $\times$ [Target - Old Estimate]

$$V_\pi(x_k) \leftarrow V_\pi(x_k) + \alpha [G_k - V_\pi(x_k)] \quad \text{Incremental form of MC estimates}$$



$$V_\pi(x_k) \leftarrow V_\pi(x_k) + \alpha \underbrace{[r_{k+1} + \gamma V_\pi(x_{k+1}) - V_\pi(x_k)]}_{\text{Temporal Difference}}$$

Instead of waiting to see all obtain rewards G_k required for computing plug in the estimate $V_\pi(x_{k+1})$

Temporal Difference

The Prediction Problem

Estimate the Value Function $V_\pi(x)$ for a given policy π .

New Estimate $\leftarrow$ Old Estimate + Step Size $\times$ [Target - Old Estimate]

$$V_\pi(x_k) \leftarrow V_\pi(x_k) + \alpha [G_k - V_\pi(x_k)] \quad \text{Incremental form of MC estimates}$$



$$V_\pi(x_k) \leftarrow V_\pi(x_k) + \alpha \underbrace{[r_{k+1} + \gamma V_\pi(x_{k+1}) - V_\pi(x_k)]}_{\text{Temporal Difference}}$$

Temporal Difference

Fundamental update rule:

$$x_k \rightarrow \underbrace{\mu_{k+1}}_{\text{update target}} = r_{k+1} + \gamma V_\pi(x_{k+1})$$

The Prediction Problem

What to do if

The state-space has many discrete elements $\text{Card}(X) \approx 10^5$

- Tabular representations quickly run out of memory
- We can not wait till we explore all states
- Need for approximation / generalization capability

The state-space is continuous

- All points can not be visited (finite time) $X = \mathbb{R}^{n_x}$
- How to handle functional representations of the value function, Q-function?

Approximation of The Value Function

Concept:

$$V_{\pi}(x_k) \approx \hat{V}_{\pi}(x_k, \theta)$$

Parameters $\theta \in \mathbb{R}^d$

Approximate value function

Approximation of The Value Function

Concept:

$$V_{\pi}(x_k) \approx \hat{V}_{\pi}(x_k, \theta)$$

Parameters $\theta \in \mathbb{R}^d$

How to optimize θ ?

Approximate value function

We need to mimic function behavior
for observed input samples x_k and
output samples μ_k



Lecture 2-5:
Function Estimation

Stochastic Gradient Method (SGD)

Cost function:
$$\text{VE}(\theta) = \sum_{x \in X} (V_{\pi}(x) - \hat{V}_{\pi}(x, \theta))^2$$

Observation:
$$x_k \rightarrow V_{\pi}(x_{k+1})$$

Gradient search:
$$\begin{aligned} \theta_{k+1} &= \theta_k - \frac{1}{2} \alpha \nabla_{\theta} \left[V_{\pi}(x_k) - \hat{V}_{\pi}(x_k, \theta_k) \right]^2 \\ &= \theta_k + \alpha \left[V_{\pi}(x_k) - \hat{V}_{\pi}(x_k, \theta_k) \right] \nabla_{\theta} \hat{V}_{\pi}(x_k, \theta_k) \end{aligned}$$

Differentiable function

Step size

Gradient

Stochastic Gradient Method for TD

Objective: $V_{\pi}(x_k) = \mathbb{E}_{\pi} \{ G_k | x_k \}$

Observation: $x_k \rightarrow \underbrace{\mu_{k+1}}_{\text{update target}} = r_{k+1} + \gamma V_{\pi}(x_{k+1}) \approx G_k$

Gradient search: $\theta_{k+1} = \theta_k + \alpha \left[r_{k+1} + \gamma \hat{V}_{\pi}(x_{k+1}, \theta_k) - \hat{V}_{\pi}(x_k, \theta_k) \right] \nabla_{\theta} \hat{V}_{\pi}(x_k, \theta_k)$

Otherwise
it can wander



Asymptotic convergence
if $\mathbb{E}_{\pi} \{ \mu_{k+1} | x_k = x \} = V_{\pi}(x)$

& $\alpha_k \rightarrow 0$
slow enough

Approximate Temporal Difference Learning

Input: the policy π to be evaluated

Parameters: step size $\alpha \in (0, 1]$

Initialize θ arbitrarily (e.g., $\theta = 0$)

for each episode **do**

 Initialize x_0

Repeat for each time step of the episode

 Obtain u_k based on x_k using policy π

 Take action u_k , observe r_{k+1}, x_{k+1}

$$\theta \leftarrow \theta + \alpha \left[r_{k+1} + \gamma \hat{V}_\pi(x_{k+1}, \theta) - \hat{V}_\pi(x_k, \theta) \right] \nabla_\theta \hat{V}_\pi(x_k, \theta)$$

$k = k + 1$

Until the states are terminal

end for

Similar algorithm
for MC and DP
(Bootstrapping)
type of methods.

Choice of Parametrization

Linear parameterization

$$\hat{V}_{\pi}(x, \theta) = \theta^{\top} \Phi(x) = \sum_{i=1}^d \theta_i \phi_i(x)$$

Features (basis functions)
 $\phi_i : X \rightarrow \mathbb{R}$



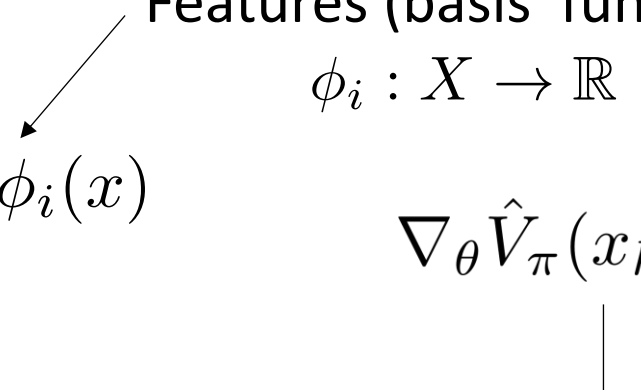
Choice of Parametrization

Linear parameterization

Features (basis functions)
 $\phi_i : X \rightarrow \mathbb{R}$

$$\hat{V}_\pi(x, \theta) = \theta^\top \Phi(x) = \sum_{i=1}^d \theta_i \phi_i(x)$$

$\nabla_\theta \hat{V}_\pi(x_k, \theta_k)$



SGD search:

$$\begin{aligned} \theta_{k+1} &= \theta_k + \alpha \left[r_{k+1} + \gamma \hat{V}_\pi(x_{k+1}, \theta_k) - \hat{V}_\pi(x_k, \theta_k) \right] \Phi(x_k) \\ &= \theta_k + \alpha \left[r_{k+1} \Phi(x_k) - \Phi(x_k) (\Phi(x_k) - \gamma \Phi(x_{k+1}))^\top \theta_k \right] \end{aligned}$$

Choice of Parametrization

Linear parameterization

$$\hat{V}_\pi(x, \theta) = \theta^\top \Phi(x) = \sum_{i=1}^d \theta_i \phi_i(x)$$

Features (basis functions)
 $\phi_i : X \rightarrow \mathbb{R}$

$\nabla_\theta \hat{V}_\pi(x_k, \theta_k)$

SGD search:

$$\begin{aligned} \theta_{k+1} &= \theta_k + \alpha \left[r_{k+1} + \gamma \hat{V}_\pi(x_{k+1}, \theta_k) - \hat{V}_\pi(x_k, \theta_k) \right] \Phi(x_k) \\ &= \theta_k + \alpha \left[r_{k+1} \Phi(x_k) - \Phi(x_k) (\Phi(x_k) - \gamma \Phi(x_{k+1}))^\top \theta_k \right] \end{aligned}$$

Consequence:

Linear param. + quadratic cost = convex problem



Guaranteed convergence
global opt. = local opt.

Choice of Parametrization

Convergence

$$\theta_{k+1} = \theta_k + \alpha \left[\underbrace{r_{k+1} \Phi(x_k)}_b - \underbrace{\Phi(x_k)(\Phi(x_k) - \gamma \Phi(x_{k+1}))^\top}_{A} \theta_k \right]$$

$$b = \mathbb{E}\{r_{k+1} \Phi(x_k)\} \quad A = \mathbb{E}\{\Phi(x_k)(\Phi(x_k) - \gamma \Phi(x_{k+1}))^\top\}$$

when converged: $\theta_k = \theta_k + \alpha \underbrace{[b - A\theta_k]}_{=0}$



Fixed point: $\theta_{\text{TD}} = A^{-1}b$

This can be also used
as an update rule (LS-TD)

must be positive definite

$$\text{VE}(\theta_{\text{TD}}) \leq \frac{1}{1-\gamma} \min_{\theta} \text{VE}(\theta)$$

Choice of Parametrization

Convergence

$$\theta_k = \theta_k + \alpha \underbrace{[b - A\theta_k]}_{=0}$$



$$\text{VE}(\theta_{\text{TD}}) \leq \frac{1}{1 - \gamma} \min_{\theta} \text{VE}(\theta)$$



Typically close to 1



Very loose upper bound

Fixed point: $\theta_{\text{TD}} = A^{-1}b$



must be positive definite

Feature Construction

Construction based on fixed selection

Polynomial features:

$$\phi_i(x) = \prod_{j=1}^{n_x} x_j^{c_{i,j}}$$

Integer in $\{0, \dots, n\}$

n^{th} -order polynomial basis

$x = (x_1 \quad \dots \quad x_{n_x})$

Number of features: $(n + 1)^{n_x}$

Easily differentiable

Example:

$$\Phi(x) = (1 \quad x_1 \quad x_2 \quad x_1 x_2 \quad x_1^2 \quad x_2^2 \quad x_1 x_2^2 \quad x_1^2 x_2 \quad x_1^2 x_2^2)$$

Feature Construction

Construction based on fixed selection

Fourier features:

$$c_i = (c_{i,1} \quad \cdots \quad c_{i,n_x})$$

$$c_{i,j} \in \{0, \dots, n\}$$

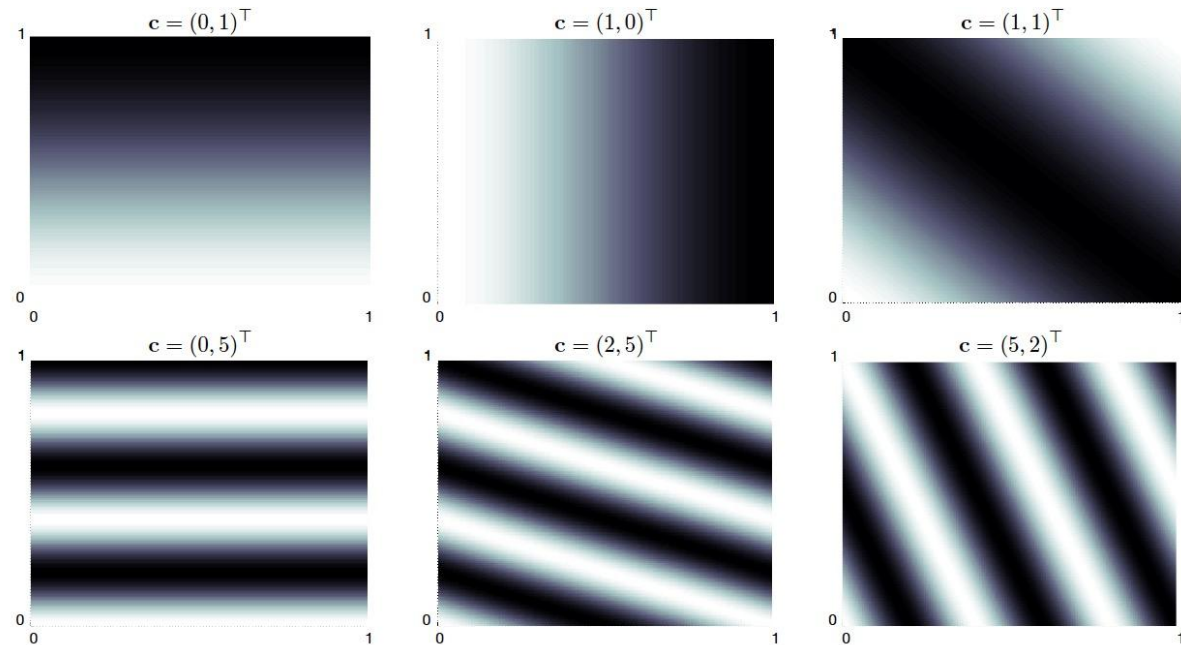
$$\phi_i(x) = \cos(\pi x^\top c_i)$$

n^{th} -order Fourier basis

Number of features:

$$(n + 1)^{n_x}$$

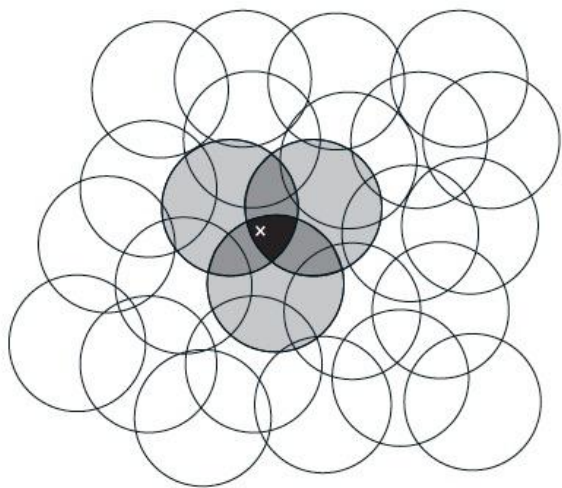
Easily differentiable



Feature Construction

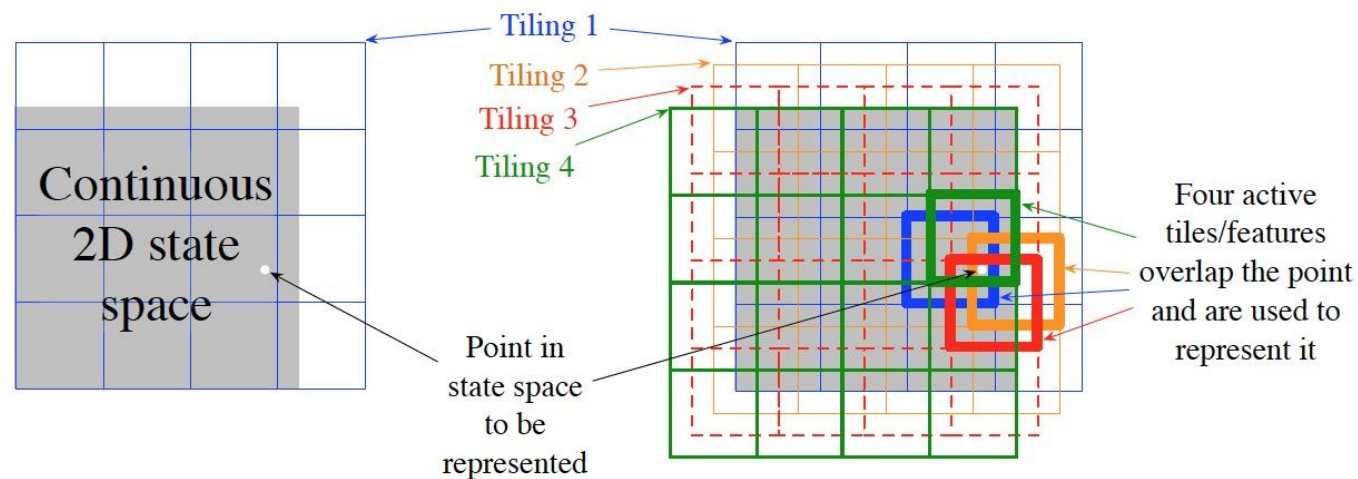
Construction based on fixed selection

Coarse coding



$$\phi_i(x) = \begin{cases} 1 & x \in \text{region } i \\ 0 & \text{else} \end{cases}$$

Tile coding



Similar to convolutional layer in ANN

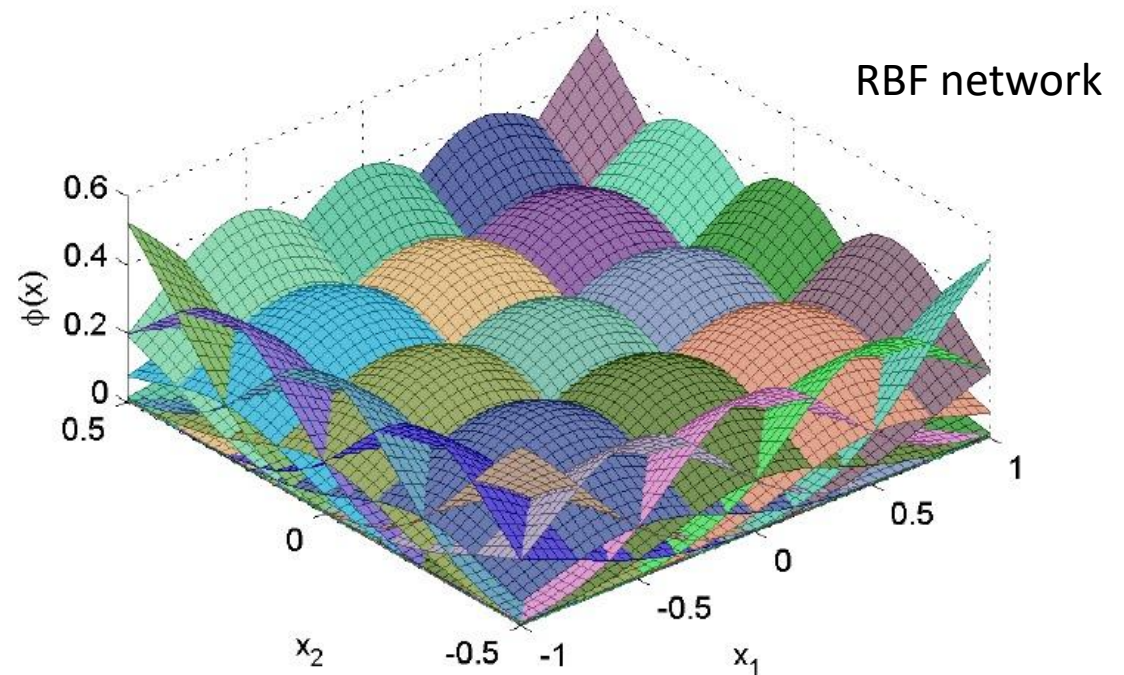
Feature Construction

Construction based on fixed selection

Kernel-based

$$\phi_i(x) = K(x, x_i)$$

↑
Kernel function
(differentiable)



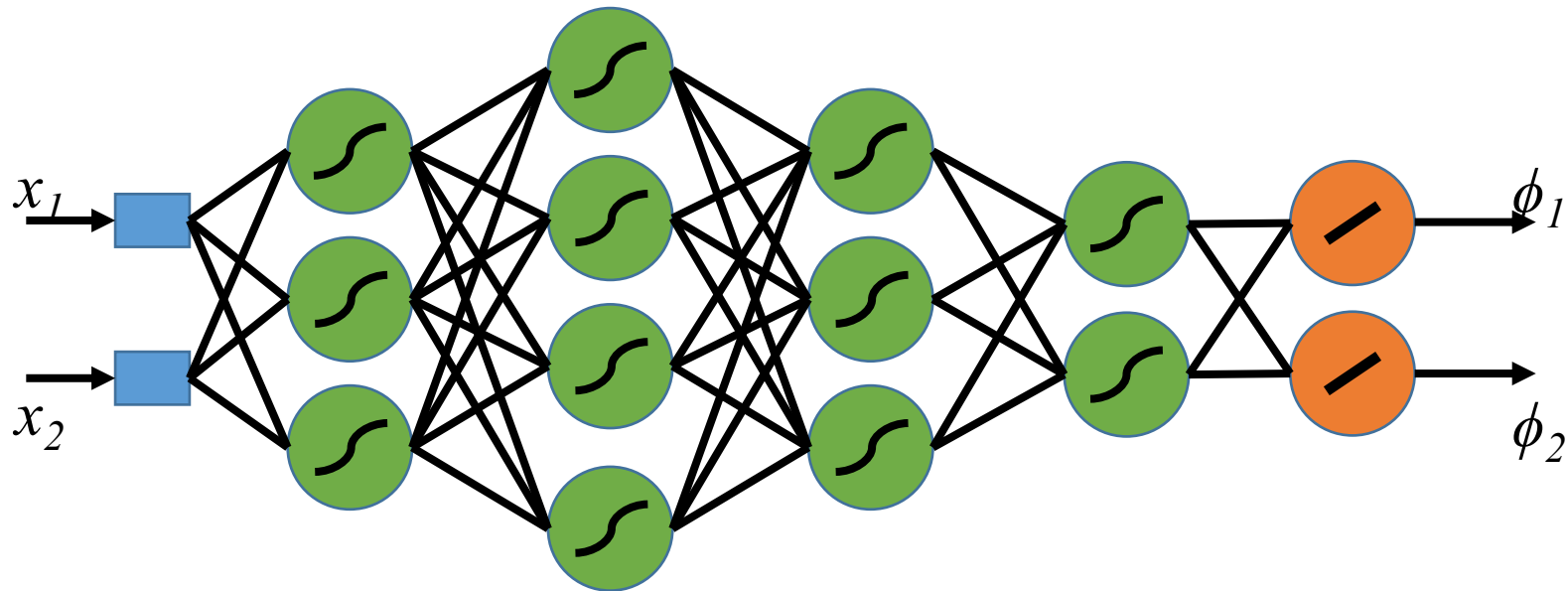
GP-based generalization (non-fixed selection):

Store the samples (or only the relevant ones: support features)

Directly apply GP (with marginalized likelihood optimization for the hyper-parameters)

Feature Construction

Construction based on ANN



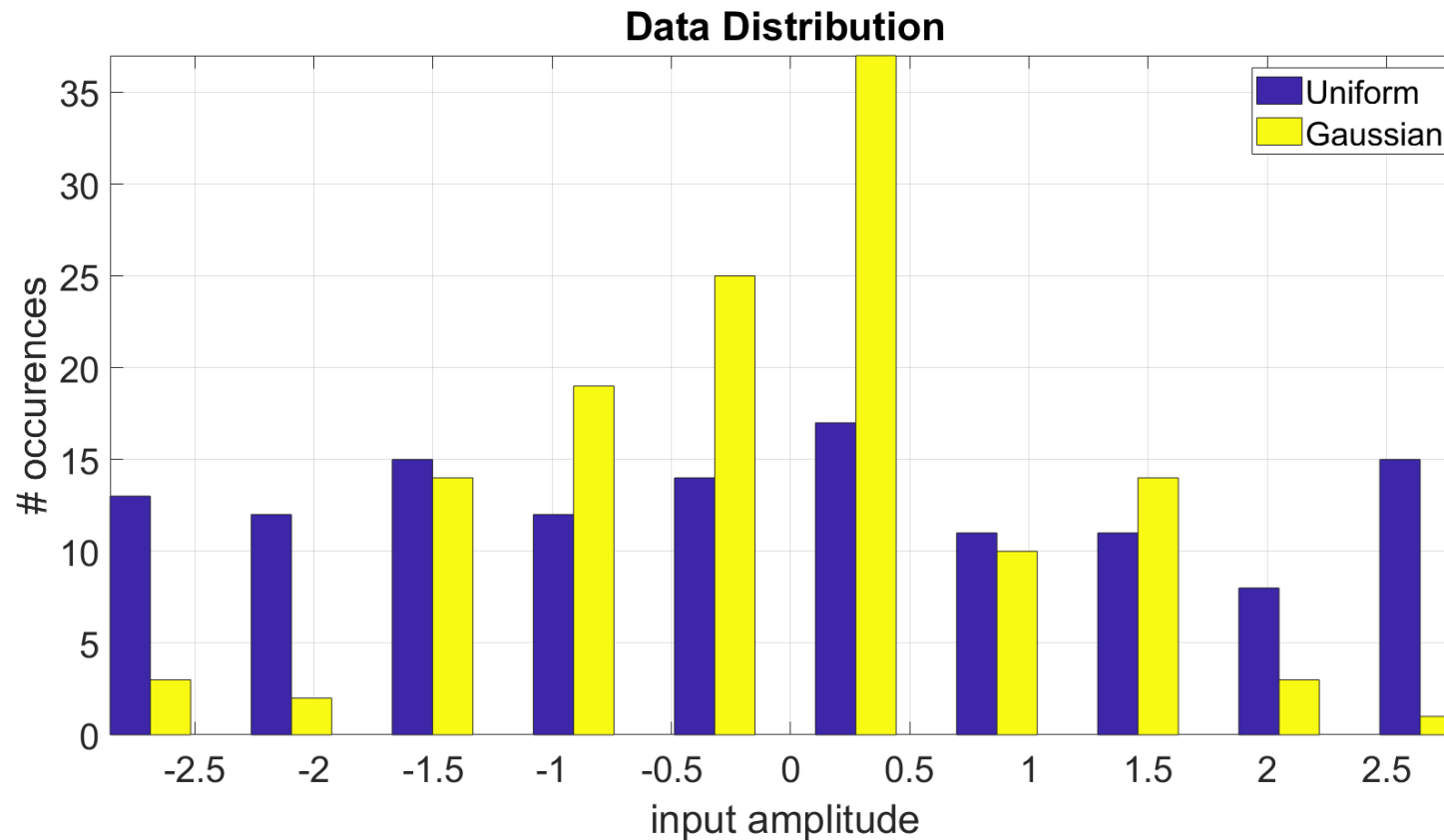
Same concepts apply as
in identification

Use backpropagation

$$\theta_{k+1} = \theta_k + \alpha \left[r_{k+1} + \gamma \hat{V}_\pi(x_{k+1}, \theta_k) - \hat{V}_\pi(x_k, \theta_k) \right] \nabla_\theta \hat{V}_\pi(x_k, \theta_k)$$

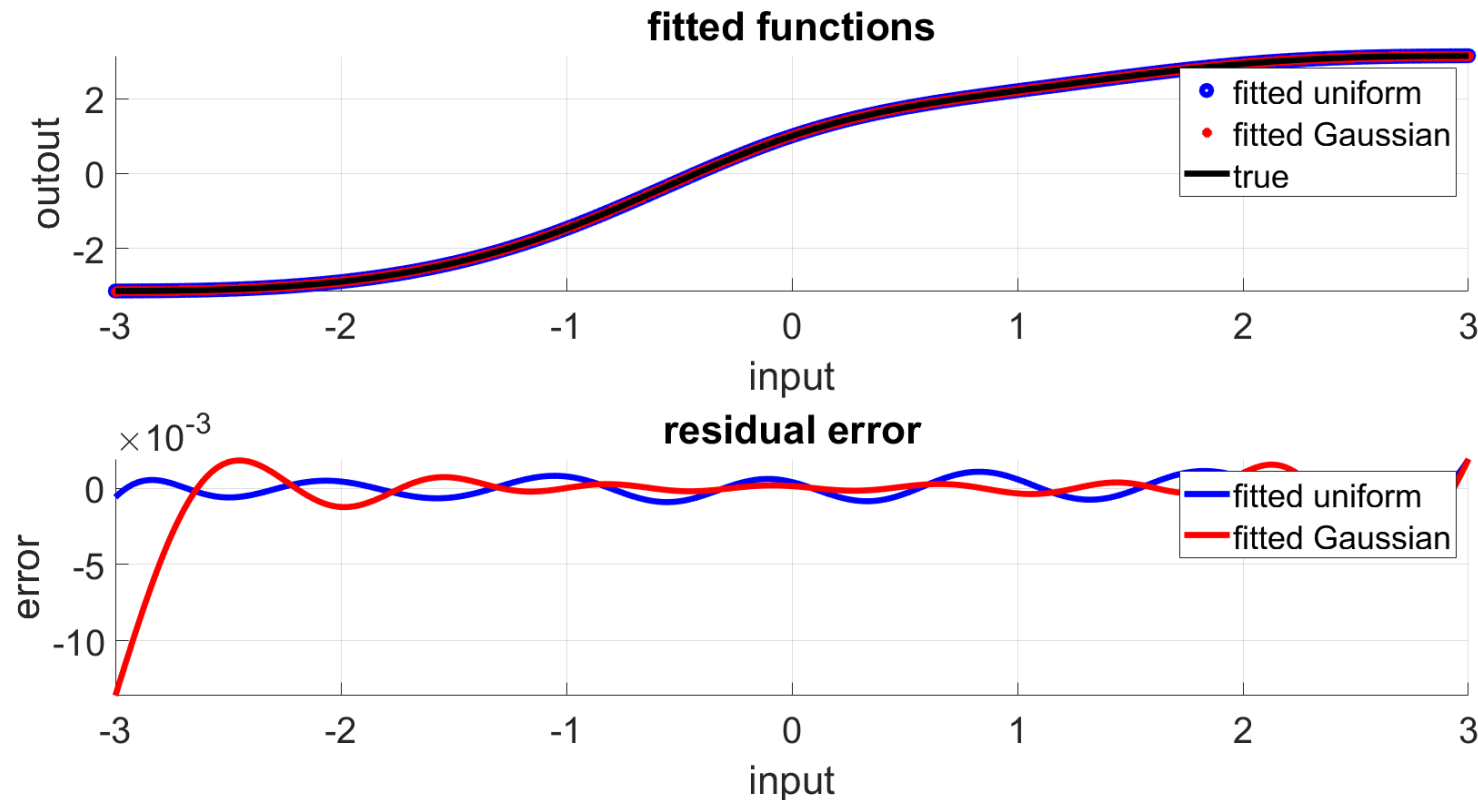
Function Approximation

Data distribution becomes important in function approximation algorithms



Function Approximation

Data distribution becomes important in function approximation algorithms



Function Approximation

Data distribution becomes important in function approximation algorithms

→ it influences how the approximation errors are spread

Can be problematic for high-dimensional state spaces

Compensate for data-distribution by using weighted cost functions

$$\text{VE}(\theta) = \sum_{x \in X} w_x \left(V_{\pi}(x) - \hat{V}_{\pi}(x, \theta) \right)^2$$

Reinforcement Learning

Overview of RL

Approximate TD Learning for Prediction

Approximate TD Learning for Control

Deep Q-Networks

Gaussian Process Q-Learning

The control problem

For a control objective, we want to estimate

Q-Function:
$$Q_{\pi}(x_k, u_k) = \mathbb{E}_{\pi} \left\{ \sum_{\tau=0}^{\infty} \gamma^{\tau} r_{k+\tau+1} \middle| x_k, u_k \right\}$$

OR

For a given policy π

Optimal Q-Function:
$$Q_{*}(x_k, u_k) = \sum_{x_{k+1}, r} p(x_{k+1}, r \mid x_k, u_k) \left(r + \gamma \max_{u_{k+1}} Q_{*}(x_{k+1}, u_{k+1}) \right)$$

The control problem

For a control objective, we want to estimate

Q-Function:
$$Q_{\pi}(x_k, u_k) = \mathbb{E}_{\pi} \left\{ \sum_{\tau=0}^{\infty} \gamma^{\tau} r_{k+\tau+1} \middle| x_k, u_k \right\}$$

OR

For a given policy π

Optimal Q-Function:
$$Q_{*}(x_k, u_k) = \sum_{x_{k+1}, r} p(x_{k+1}, r \mid x_k, u_k) \left(r + \gamma \max_{u_{k+1}} Q_{*}(x_{k+1}, u_{k+1}) \right)$$

TO FIND

Optimal policy:
$$\pi_{*}(x_k) = \arg \max_{u_k} Q_{*}(x_k, u_k)$$

Greedy in $Q_{*}(x_k, u_k)$

Types of RL Control

By path to optimal solution

- **Off-policy** – find Q_* , use it to compute π_*
- **On-policy** – find Q_π , improve π , repeat

By level of interaction with the process

- **Online** – learn by interacting with the process
- **Offline** – data collected in advance (Monte-Carlo methods)

By model knowledge

- **Model-free** – no p , only transition data (standard RL)
- **Model-based** – p is known (Dynamic Programming)
- **Model-learning** – estimate p from transition data

Q-Learning (off policy, online, model-free)

Take **Bellman optimality equation** at some state and action

$$Q_*(x_k, u_k) = \sum_{x_{k+1}, r} p(x_{k+1}, r | x_k, u_k) \left(r + \gamma \max_{u_{k+1}} Q_*(x_{k+1}, u_{k+1}) \right)$$

Turn into **iterative update**

$$Q(x_k, u_k) \leftarrow \sum_{x_{k+1}, r} p(x_{k+1}, r | x_k, u_k) \left(r + \gamma \max_{u_{k+1}} Q(x_{k+1}, u_{k+1}) \right)$$

Instead of a transition model, use the **transition sample** at each step

$$Q(x_k, u_k) \leftarrow r_{k+1} + \gamma \max_{u_{k+1}} Q(x_{k+1}, u_{k+1})$$

Q-Learning (off policy, online, model-free)

Implemented with incremental update

$$Q(x_k, u_k) \leftarrow Q(x_k, u_k) + \alpha \left(\underbrace{r_{k+1} + \gamma \max_{u_{k+1}} Q(x_{k+1}, u_{k+1}) - Q(x_k, u_k)}_{\text{Temporal Difference}} \right)$$

Learning rate
 $\alpha \in (0, 1]$

How to use this over continuous spaces?

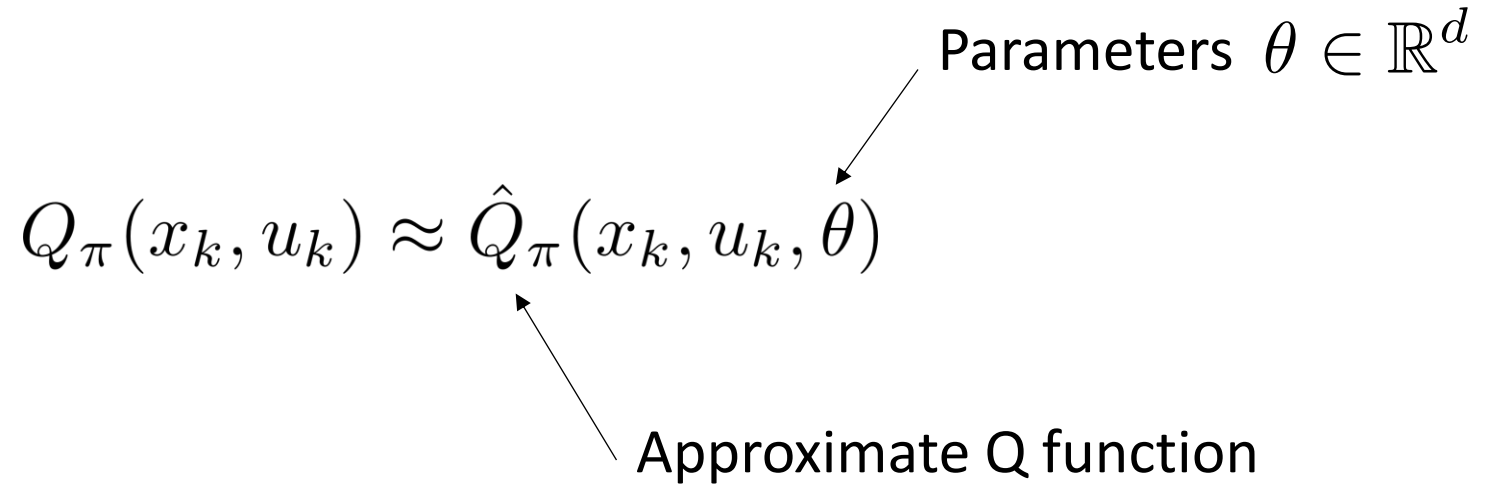
Approximation of The Q Function

Concept:

$$Q_{\pi}(x_k, u_k) \approx \hat{Q}_{\pi}(x_k, u_k, \theta)$$

Parameters $\theta \in \mathbb{R}^d$

Approximate Q function



Policy computation:

$$\pi(x_k) = \arg \max_{u_k} \hat{Q}(x_k, u_k, \theta) \quad (\text{greedy policy})$$

Approximation of The Q Function

Parametrization of $\hat{Q}(x, u, \theta)$:

- Must be **differentiable** with simple computation of the gradient, like

$$\hat{Q}(x, u, \theta) = \sum_{i=1}^d \theta_i \phi_i(x, u) \quad (\text{linear parametrization})$$

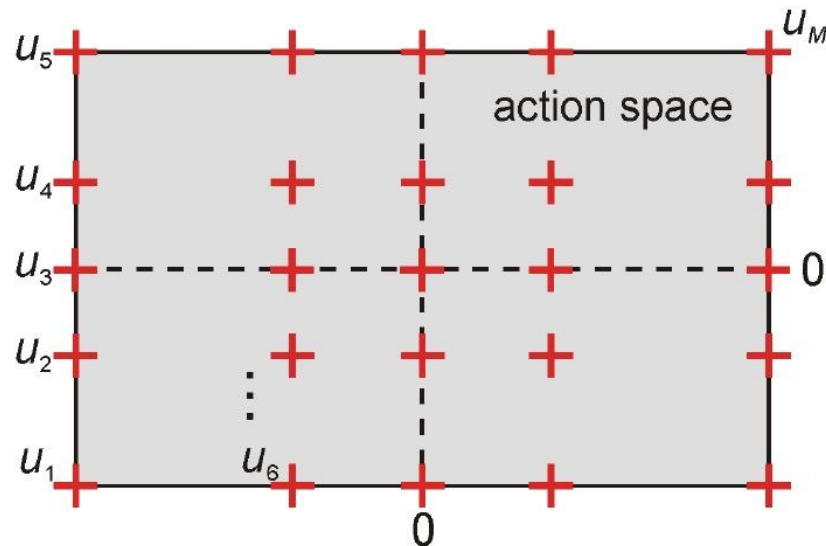
- Policy optimization should be simple

$$\pi(x) = \arg \max_u \sum_{i=1}^d \theta_i \phi_i(x, u) \quad (\text{cont. optimization})$$

Approximation of The Q Function

Parametrization of $\hat{Q}(x, u, \theta)$:

- Feature construction: same as for prediction
- Policy optimization: typically, **action discretization** is used:



Action space is gridded:

$$u_1, \dots, u_M \in U$$

Policy computation:

$$\pi(x_k) = \arg \max_{u \in \{u_j\}_{j=1}^M} \hat{Q}(x_k, u, \theta)$$

(simple max operation)

Approximate Q-Learning

Based on SGD:

$$\theta_{k+1} = \theta_k + \alpha \left[\underbrace{r_{k+1} + \gamma \max_{u_{k+1}} \hat{Q}(x_{k+1}, u_{k+1}, \theta) - \hat{Q}(x_k, u_k, \theta)}_{\text{Temporal Difference}} \right] \nabla_{\theta} \hat{Q}(x_k, u_k, \theta)$$

Learning rate
 $\alpha \in (0, 1]$

Approximate Q-Learning

Parameters: step size $\alpha \in (0, 1]$ and $0 < \epsilon < 1$

Initialize $\hat{Q}(x, u, \theta)$ arbitrarily (e.g., $\theta = 0$)

for each episode **do**

Initialize x_0

Repeat for each time step of the episode

Obtain u_k based on x_k using policy π derived from $\hat{Q}$

(e.g., ϵ -greedy) **common choice**

Take action u_k , observe r_{k+1}, x_{k+1}

$$\theta_{k+1} = \theta_k + \alpha \left[r_{k+1} + \gamma \max_u \hat{Q}(x_{k+1}, u, \theta) - \hat{Q}(x_k, u_k, \theta) \right] \nabla_{\theta} \hat{Q}(x_k, u_k, \theta)$$

$k = k + 1$

Until the states are terminal

end for

Approximate Q-Learning

Properties

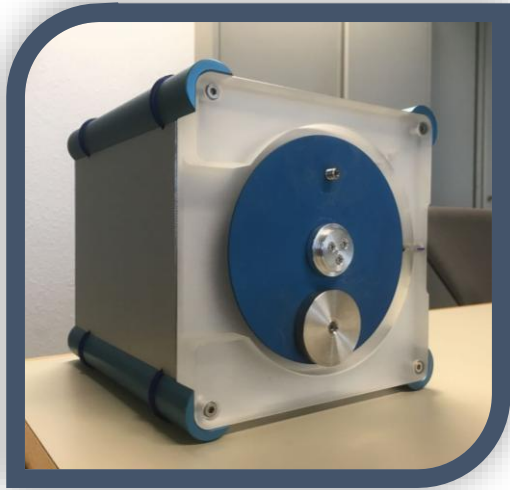
On-policy approximate methods: convergence can be achieved

- The promise of future reward must be kept, and the corresponding action is needed to be taken for convergence.

Off-policy approximate methods: no convergence guarantees

- Promise of future reward is made, but then another action might follow and the promise, including its error, is forgotten.
- Approximate Q-learning with ϵ -greedy policy can diverge, but usually works well in practice!

Example

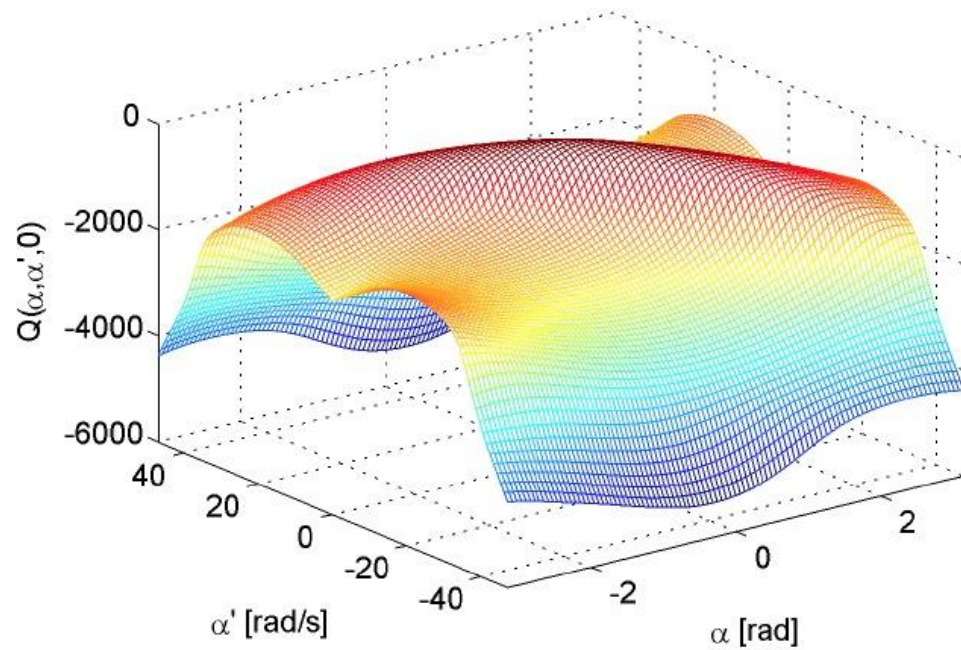


- State: $x = (\alpha \quad \dot{\alpha})$ (angle and velocity)
 - Input: $u = \text{voltage}$
 - Reward: $r_{k+1} = \rho(x_k, u_k) = -x_k^\top \begin{bmatrix} 5 & 0 \\ 0 & 0.1 \end{bmatrix} x_k - u_k^\top u_k$
 - Discount: $\gamma = 0.98$
-
- Objective: stabilize top position (swing up)
 - Insufficient actuation (needs to swing back & forth)

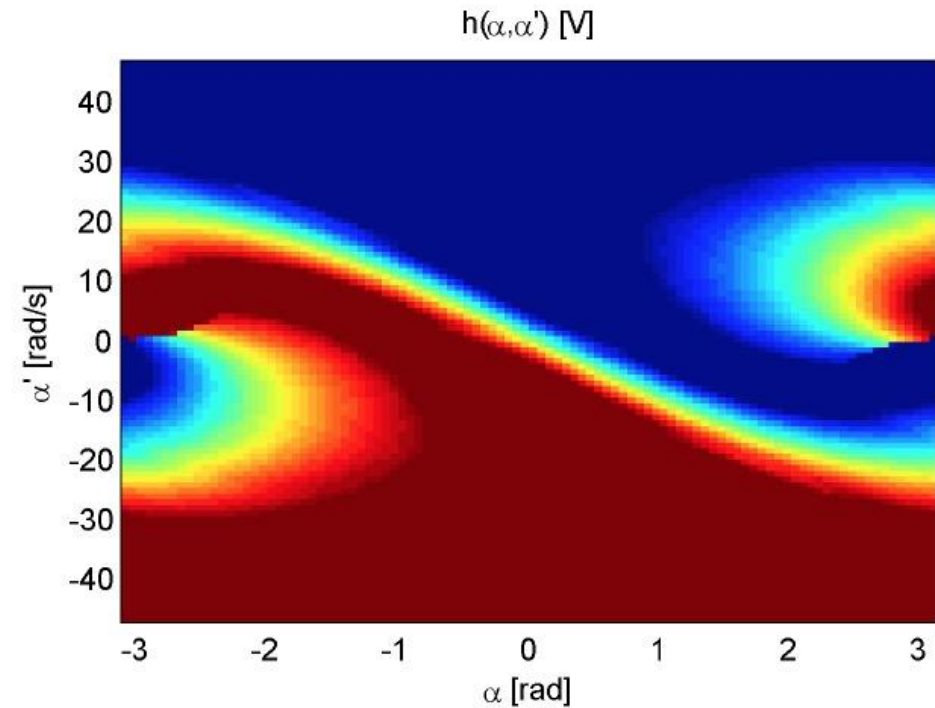
Example

[See video](#)

Q-function for $u = 0$



Policy $\pi(\alpha, \dot{\alpha})$

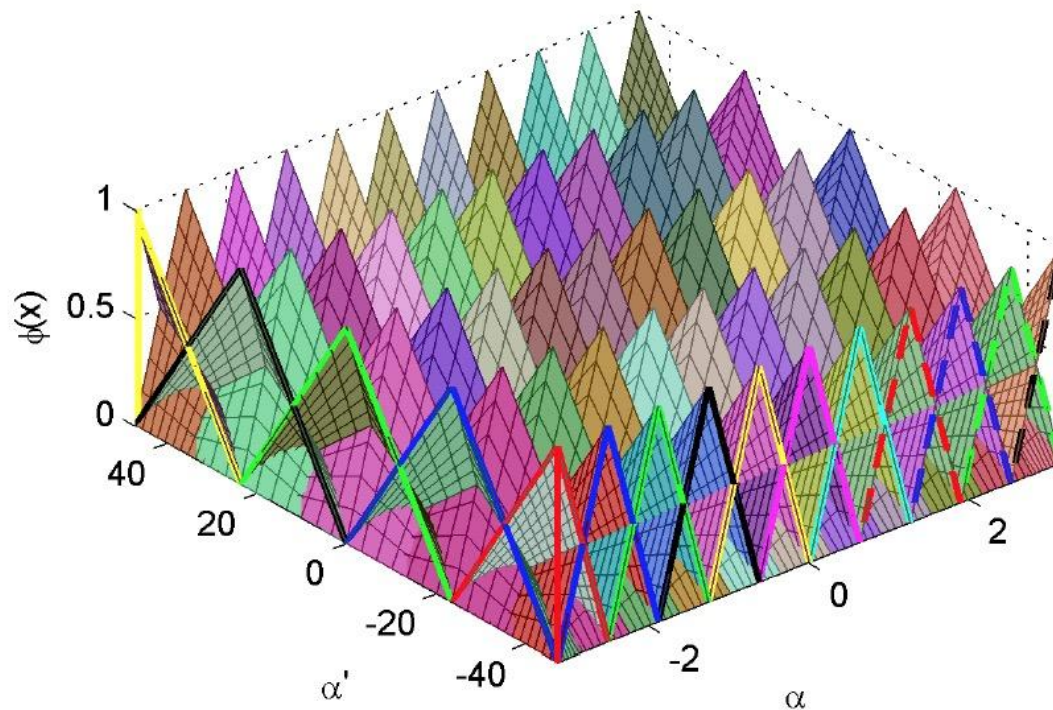


Near optimal solution

Example

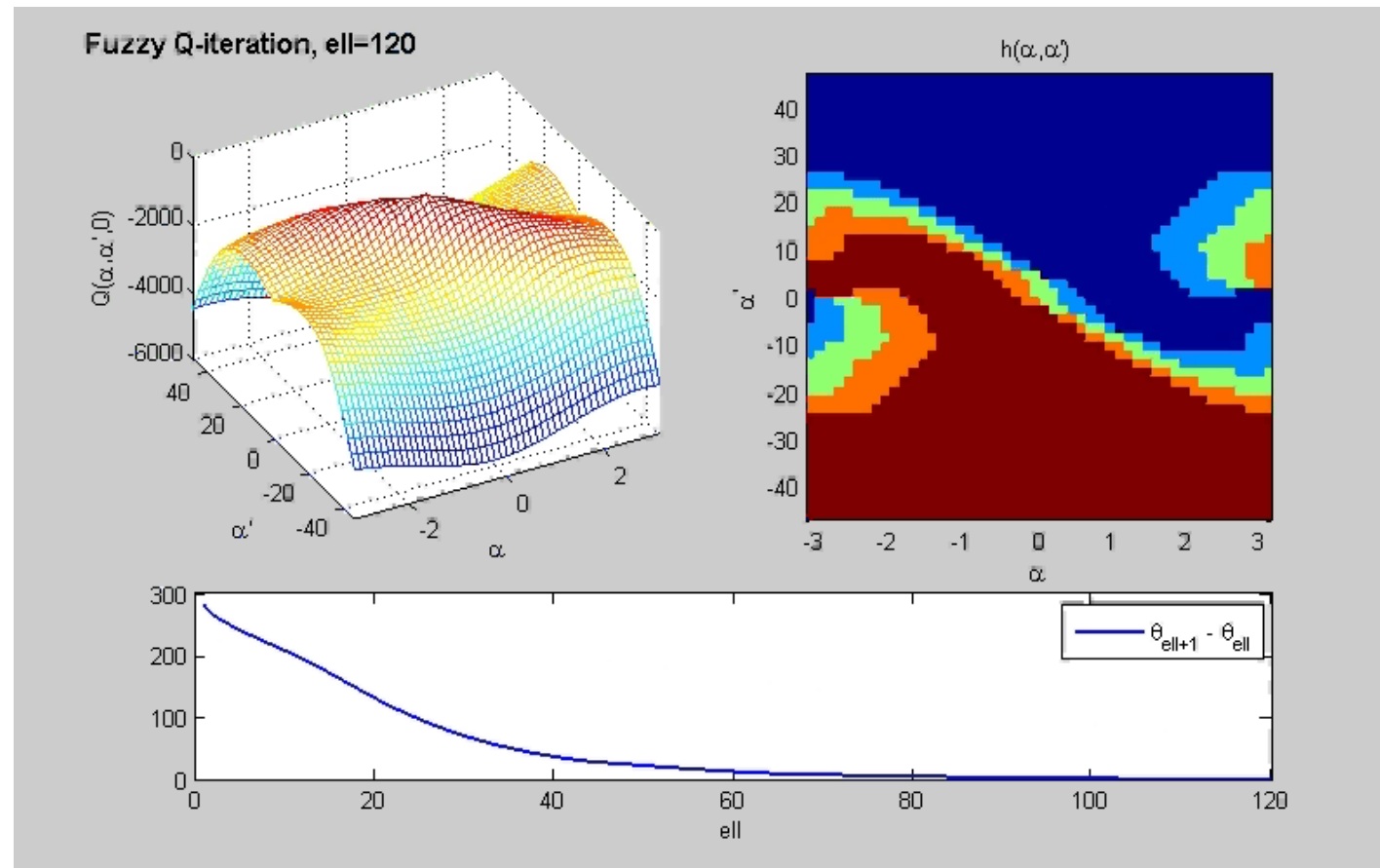
Features: Triangular membership functions: 41x21 equidistant grid

Input space: 5 actions, log-placed around 0



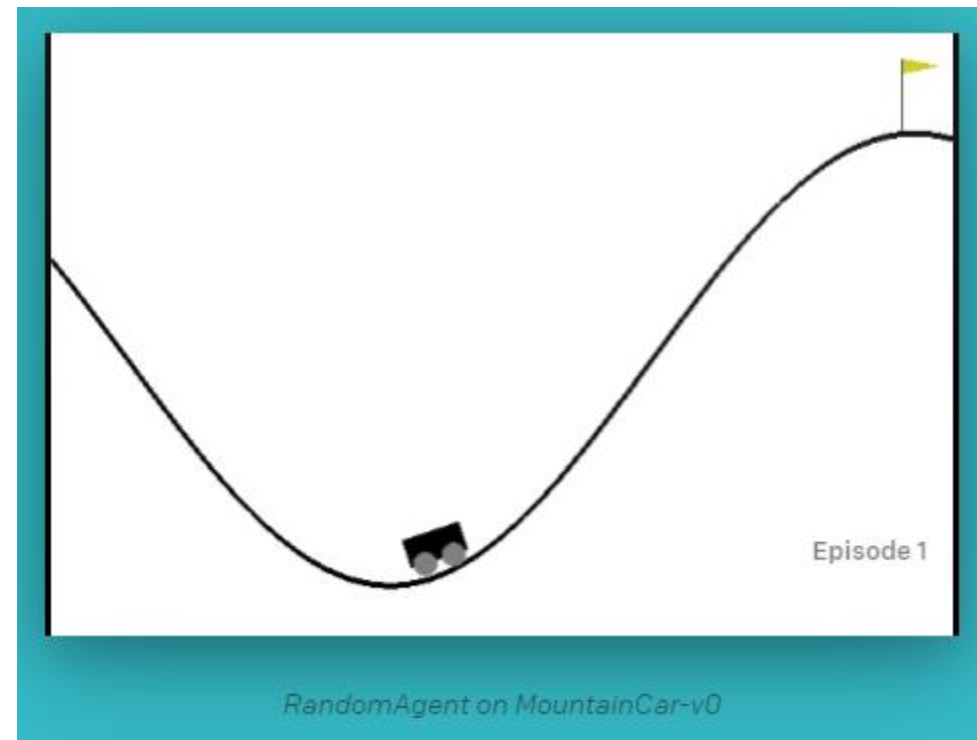
Example

Results:



Mountain Car

A car is on a one-dimensional track, positioned between two "mountains". The goal is to drive up the mountain on the right; however, the car's engine is not strong enough to scale the mountain in a single pass. Therefore, the only way to succeed is to drive back and forth to build up momentum.



Reinforcement Learning

Overview of RL

Approximate TD Learning for Prediction

Approximate TD Learning for Control

Gaussian Process Q-Learning

Deep Q-Networks

Reinforcement Learning

Overview of RL

Approximate TD Learning for Prediction

Approximate TD Learning for Control

Deep Q-Networks

Gaussian Process Q-Learning

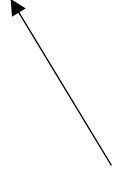
Deep Q Networks (DQN)

$$Q_{\pi}(x_k, u_k) \approx \hat{Q}_{\pi}(x_k, u_k, \theta)$$

Parameters



Approximate Q function
with deep neural net

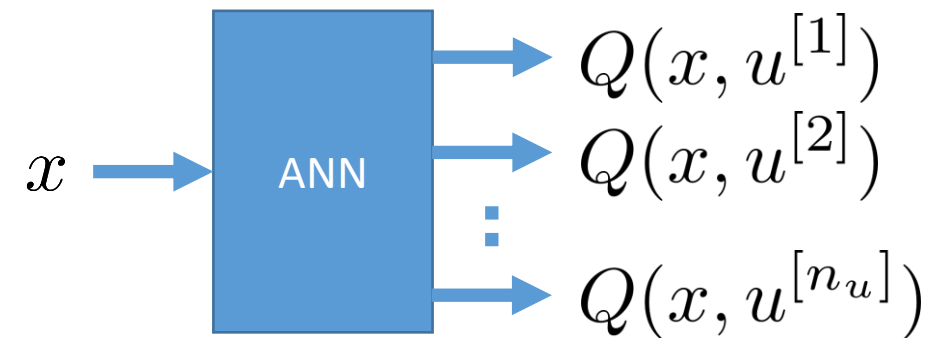
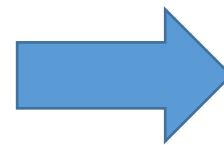
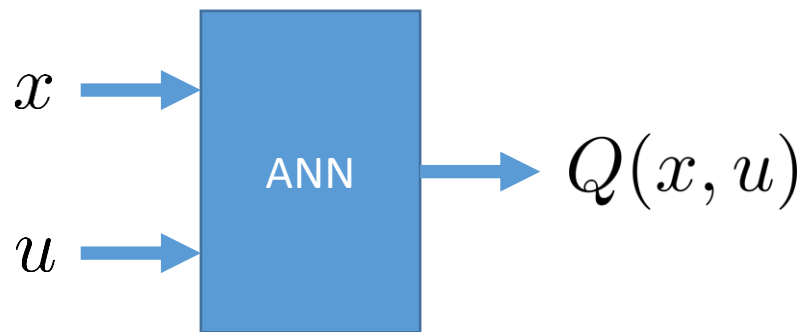


Deep Q Networks (DQN)

$$Q_{\pi}(x_k, u_k) \approx \hat{Q}_{\pi}(x_k, u_k, \theta)$$

Parameters

Approximate Q function
with deep neural net



Requires network evaluation for each possible action

Requires only one network evaluation

Deep Q Networks (DQN)

$$\theta_{k+1} = \theta_k + \alpha \left[r_{k+1} + \gamma \max_{u_{k+1}} \hat{Q}(x_{k+1}, u_{k+1}, \theta) - \hat{Q}(x_k, u_k, \theta) \right] \nabla_{\theta} \hat{Q}(x_k, u_k, \theta)$$

Learning rate

$$\alpha \in (0, 1]$$

Temporal Difference

↓
Backpropagation

Deep Q Networks (DQN)

$$\theta_{k+1} = \theta_k + \alpha \left[\underbrace{r_{k+1} + \gamma \max_{u_{k+1}} \hat{Q}(x_{k+1}, u_{k+1}, \theta) - \hat{Q}(x_k, u_k, \theta)}_{\text{Non-stationary target}} \right] \nabla_{\theta} \hat{Q}(x_k, u_k, \theta)$$

Non-stationary target

Challenge: Non-stationary target

The target is continuously changing with each iteration. In deep learning, the target variable does not change and hence the training is stable, which is just not true for RL.

Challenge: Correlated trajectories

Samples are received from a trajectory of a dynamical system; hence the data is strongly correlated over (short) timeframes.

Break the Correlation: Experience Replay

Idea: Introduce a memory that stores pervious [action - state transitions - reward] values

Usage: Sample mini-batches from the memory to compute the gradient at each optimization iteration

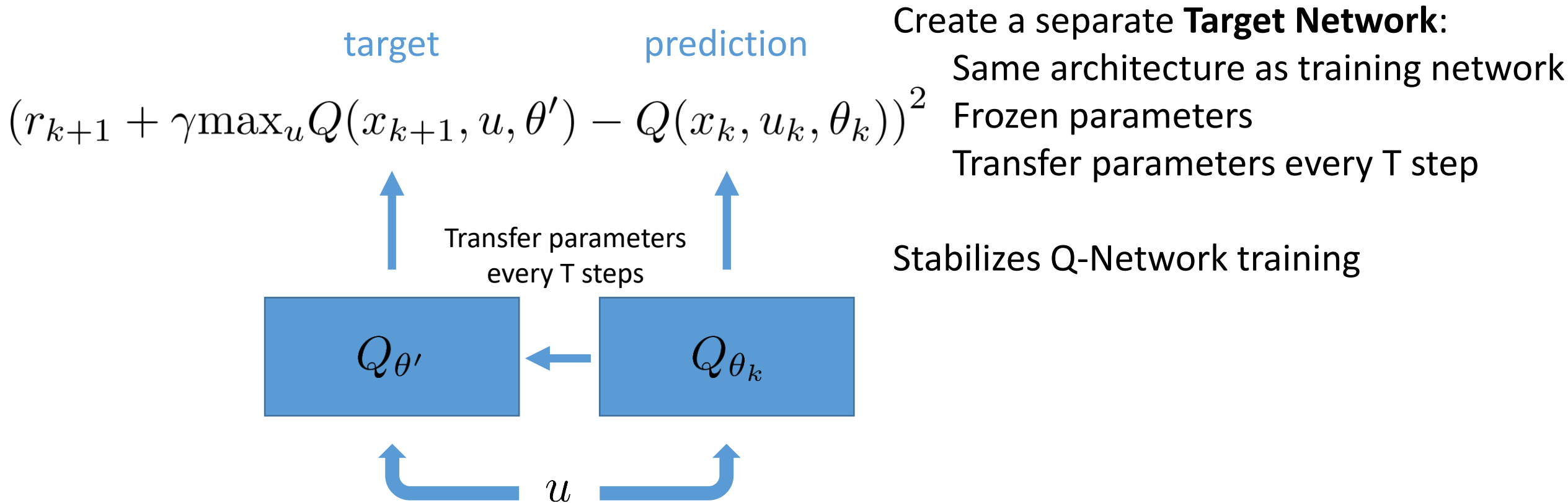
Result:

When replay *memory is large* → experience replay is *close to sampling independent transitions* from an explorative policy.

This *reduces the variance of the gradient*, which is used to update θ .

Experience replay *stabilizes the training of DQN*, which benefits the algorithm in terms of computation.

Solve the Non-Stationarity: Target Network



Bias-Variance Trade-Off of the Bellman Error

$$Y_k = r_{k+1} + \gamma \max_u Q_{\theta^*}(x_{k+1}, u)$$

Targets

$$\text{VE}(\theta) = \frac{1}{n} \sum_{i=1}^n (Y_i - Q_{\theta}(x_i, u_i))^2$$

Cost

Bias-Variance Trade-Off of the Bellman Error

$$Y_k = r_{k+1} + \gamma \max_u Q_{\theta^*}(x_{k+1}, u)$$

Targets

$$\text{VE}(\theta) = \frac{1}{n} \sum_{i=1}^n (Y_i - Q_{\theta}(x_i, u_i))^2$$

Cost

Bias-Variance Decomposition

$$\mathbb{E}\{\text{VE}(\theta)\} = \|Q_{\theta} - TQ_{\theta}\|^2 + \mathbb{E}\left\{[Y_1 - (TQ_{\theta})(x_1, u_1)]^2\right\}$$

$$(TQ)(x_k, u_k) = r_{k+1} + \gamma \mathbb{E} \left\{ \max_u Q(x_{k+1}, u) \middle| x_k, u_k \right\}$$

Bellman operator
(see previous lecture)

$$(TQ^*) = Q^*$$

Bellman operator applied on optimal Q function = optimal Q function

Bias-Variance Trade-Off of the Bellman Error

Without Target Network

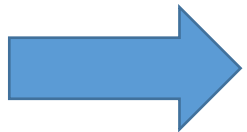
Bias-Variance Decomposition

$$\mathbb{E}\{\text{VE}(\theta)\} = \|Q_\theta - TQ_\theta\|^2 + \mathbb{E}\left\{[Y_1 - (TQ_\theta)(x_1, u_1)]^2\right\}$$

Mean Squared Bellman Error
(MSBE)

Variance of Y_1

Both depend on θ !



Minimizing the cost function is different than minimizing the MSBE

Bias-Variance Trade-Off of the Bellman Error

Without Target Network

Bias-Variance Decomposition

$$\mathbb{E}\{\text{VE}(\theta)\} = \|Q_\theta - TQ_\theta\|^2 + \mathbb{E}\left\{[Y_1 - (TQ_\theta)(x_1, u_1)]^2\right\}$$

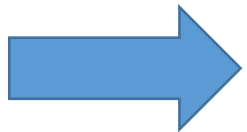
What we actually
want to minimize
since for the
optimal Q function
it holds that:

$$(TQ^*) = Q^*$$

Mean Squared Bellman Error
(MSBE)

Variance of Y_1

Both depend on θ !



Minimizing the cost function is different than minimizing the MSBE

Bias-Variance Trade-Off of the Bellman Error

With Target Network

Bias-Variance Decomposition

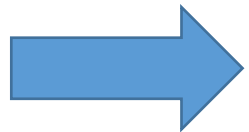
$$\mathbb{E}\{\text{VE}(\theta)\} = \|Q_\theta - TQ_{\theta'}\|^2 + \mathbb{E}\left\{[Y_1 - (TQ_{\theta'})(x_1, u_1)]^2\right\}$$

Mean Squared Bellman Error
(MSBE)

dependent on θ

Variance of Y_1

independent from θ



Minimizing the cost function is close to solving

$$\text{minimize}_\theta \|Q_\theta - TQ_{\theta'}\|^2$$

Bias-Variance Trade-Off of the Bellman Error

With Target Network

Bias-Variance Decomposition

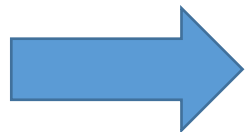
$$\mathbb{E}\{\text{VE}(\theta)\} = \|Q_\theta - TQ_{\theta'}\|^2 + \mathbb{E}\left\{[Y_1 - (TQ_{\theta'})(x_1, u_1)]^2\right\}$$

Mean Squared Bellman Error
(MSBE)

dependent on θ

Variance of Y_1

independent from θ



Minimizing the cost function is close to solving

$$\text{minimize}_\theta \|Q_\theta - TQ_{\theta'}\|^2$$

By iteratively (but slowly) updating θ' we hope to converge to:
 $(TQ^*) = Q^*$

DQN Algorithm

Input: a family of deep Q-networks Q_θ

Parameters: stepsize α , exploration probability $\epsilon \in (0, 1)$

Parameters: update freq. T_{target} , minibatch size n , replay memory $\mathcal{M}$

Initialize θ arbitrarily (e.g., $\theta = 0$)

Initialize the replay memory $\mathcal{M}$ to be empty

for each episode **do**

 Initialize x_0

Repeat for each time step of the episode

 Obtain u_k based on x_k using policy π derived from $\hat{Q}$
(e.g., ϵ -greedy)

 Take action u_k , observe r_{k+1}, x_{k+1} replay

 Store transition $(x_k, a_k, r_{k+1}, x_{k+1})$ in $\mathcal{M}$

 Sample random minibatch $(x_i, u_i, r_{i+1}, x_{i+1})_{i \in [n]}$ from $\mathcal{M}$ minibatch SGD

$\theta_{k+1} = \theta_k + \alpha \frac{1}{n} \sum_{i=1}^n \left[r_{i+1} + \gamma \max_u \hat{Q}(x_{i+1}, u, \theta') - \hat{Q}(x_i, u_i, \theta_k) \right] \nabla_{\theta} \hat{Q}(x_i, u_i, \theta_k)$

 every T_{target} steps: $\theta' \leftarrow \theta_{k+1}$ target network update

$k = k + 1$

Until the states are terminal

end for

Reinforcement Learning

Overview of RL

Approximate TD Learning for Prediction

Approximate TD Learning for Control

Deep Q-Networks

Gaussian Process Q-Learning

Gaussian-Process based Q Learning

Idea in the literature¹:

- GPs offer a probabilistic function approximation framework
- GP uncertainty can be used for efficient exploration

¹ e.g.: Maximilian Bloor et al. Gaussian Process Q-Learning for Finite-Horizon Markov Decision Processes. Reinforcement Learning Conference 2025, Montréal, Canada. <https://openreview.net/pdf?id=i9VYBEhX5z>

Gaussian-Process based Q Learning

Algorithm Overview

0. Initialize the GP-based Q-function estimate
1. Generate data using a policy rollout
2. Update the estimated Q-Values
3. Update the GP Q-function estimate
4. Go back to step 1

Gaussian-Process based Q Learning

Initialize the GP-based Q-function estimate

- Q-function is the realization of a GP:

$$\begin{bmatrix} \mathbf{Q} \\ Q(z_*) \end{bmatrix} \approx \left(0, \begin{bmatrix} K_{zz} & K_z(z_*) \\ K_z^\top(z_*) & k(z_*, z_*) \end{bmatrix} \right)$$

where: $z = [x \ u]^\top$, $\mathbf{Q}$ are stored Q-value observations

$$\hat{Q}(z_*) = K_z^\top(z_*) K_{zz}^{-1} \mathbf{Q}$$

$$\hat{c}_Q(z_*) = k(z_*, z_*) - K_z^\top(z_*) K_{zz}^{-1} K_z(z_*)$$

posterior estimate

- Use sparse GP to keep computational tractable

Gaussian-Process based Q Learning

Policy Rollout – Thompson Sampling

Rollout length of T samples

For each time-step select action through Thompson sampling:

Obtain the policy by sampling a Q-function from the Gaussian Process distribution

$$\pi(x) = \operatorname{argmax}_u Q_s(x, u)$$

$$Q_s(x, u) \sim \mathcal{N}(\hat{Q}, \hat{c}_Q)$$

Take action, store action, state, reward information

Move to next time step

GP-based exploration-exploitation tradeoff!

Gaussian-Process based Q Learning

Update the estimated Q-Values

$$\mathbf{Q}(x_k, u_k) \leftarrow \hat{Q}(x_k, u_k) + \alpha \left(r_{k+1} + \gamma \max_{u_{k+1}} \hat{Q}(x_{k+1}, u_{k+1}) - \hat{Q}(x_k, u_k) \right)$$

$$\forall k < T - 1$$

Gaussian-Process based Q Learning

Update the GP Q-function estimate

Starting from the updated Q-Value observations $\mathbf{Q}$

$$\hat{Q}(z_*) = K_z^\top(z_*)K_{zz}^{-1}\mathbf{Q}$$

$$\hat{c}_Q(z_*) = k(z_*, z_*) - K_z^\top(z_*)K_{zz}^{-1}K_z(z_*)$$

Gaussian-Process based Q Learning

Algorithm Overview

0. Initialize the GP-based Q-function estimate
1. Generate data using a policy rollout
2. Update the estimated Q-Values
3. Update the GP Q-function estimate
4. Go back to step 1

Reinforcement Learning

Overview of RL

Approximate TD Learning for Prediction

Approximate TD Learning for Control

Gaussian Process Q-Learning

Deep Q-Networks

Perspectives

There are many alternative methods for approximate learning

- On-policy methods
- Policy gradient methods
- Actor-critic methods

Fundamental dilemma:

- Efficiency of DP: Models make it possible to plan and synthesize policy, otherwise we only rely on experience. Experiments are costly and risky.
- Efficiency of RL: Experiments allow to explore and improve exploitation on the long run. Models are inherently uncertain.
- How to have a working marriage of DP and RL?